Démo TinyOS + Cooja sous Linux

Ce guide montre comment compiler un petit programme TinyOS et le faire tourner dans le simulateur Cooja. Les capteurs envoient une température par radio, une passerelle la reçoit et lève une alerte au-dessus de 90 °C. Compte environ 20 minutes si les outils sont déjà installés.

Ce qu'il te faut

- Un PC sous **Ubuntu** (testé sur Ubuntu 20.04), ou une machine virtuelle Ubuntu.
- L'archive `demo_tinyos_cooja_groupe7.zip`, envoyée avec ce guide.
- TinyOS, le compilateur nesC, le compilateur MSP430, Java et Contiki-NG (qui contient Cooja). L'étape 0 vérifie s'ils sont présents. Sinon, l'annexe explique comment les installer.

Le principe en une image

Le code est écrit en **nesC**, le langage de TinyOS. La compilation produit un binaire pour la carte **TelosB**, appelée « Sky » dans Cooja. Cooja charge ce binaire et l'exécute exactement comme le ferait la vraie carte.

```
DemoUsineC.nc + DemoUsineAppC.nc     ← le code, en nesC
        │  make telosb
        ▼
   nescc       →  app.c              ← nesC est traduit en C
   msp430-gcc  →  main.exe           ← binaire pour le processeur de la carte
        │  copié sous le nom demo_usine.sky
        ▼
   Cooja, type de carte « Sky mote »  ← charge et exécute ce binaire
        ▼
   LED, radio et messages affichés en temps réel
```

Étape 0 · Vérifier les outils

Ouvre un terminal (**Ctrl+Alt+T**) et tape :

```
nescc --version        # compilateur nesC (TinyOS)
msp430-gcc --version   # compilateur pour le processeur des cartes Sky/TelosB
java -version          # Java 17 ou plus récent
ls ~/tinyos-main ~/contiki-ng/tools/cooja
```

Si une commande répond « command not found » ou « No such file », passe d'abord par l'**annexe**.

TinyOS doit savoir où il est installé. Vérifie :

```
grep TINYOS_ROOT_DIR ~/.bashrc
```

Si rien ne s'affiche, ajoute la ligne puis recharge :

```
echo 'export TINYOS_ROOT_DIR=$HOME/tinyos-main' >> ~/.bashrc
source ~/.bashrc
```

Étape 1 · Décompresser la démo

```
mkdir -p ~/projet_g7
unzip demo_tinyos_cooja_groupe7.zip -d ~/projet_g7
cd ~/projet_g7/simulation
sed -i 's/\r$//' 01_demo_tinyos_cooja/*.sh outils/*.sh   # retire les fins de ligne Windows
chmod +x 01_demo_tinyos_cooja/*.sh outils/*.sh
```

Tu obtiens :

```
~/projet_g7/simulation/01_demo_tinyos_cooja/   # code nesC, scénario Cooja, scripts
~/projet_g7/simulation/outils/cooja.sh         # lanceur de Cooja
```

Étape 2 · Compiler le programme TinyOS

```
cd ~/projet_g7/simulation/01_demo_tinyos_cooja
bash construire.sh
```

Ce script :

1. copie le code dans `~/projets/01_demo_tinyos_cooja` ;
2. lance `make telosb` (nesC → C → binaire MSP430) ;
3. copie `build/telosb/main.exe` sous le nom `demo_usine.sky` ;
4. prépare le scénario Cooja `demo_usine.csc` avec le bon chemin vers ce binaire.

À la fin, tu dois voir la taille du programme (environ 12 Ko) et `OK : …/demo_usine.sky`. Un avertissement « LOW POWER COMMUNICATIONS DISABLED » est normal.

Étape 3 · Ouvrir Cooja avec le scénario

```
bash ouvrir_cooja.sh
```

La fenêtre met environ 15 s à s'ouvrir, plus longtemps la toute première fois. Clique sur **Start** en haut.

Sans le script, la commande équivalente est :

```
cd ~/contiki-ng/tools/cooja
./gradlew run --args="$HOME/projets/01_demo_tinyos_cooja/demo_usine.csc"
```

Ce que tu dois voir

FENÊTRE	CONTENU
Network	Les 4 cartes à leur position. Le nœud 1 est le puits (la passerelle), les nœuds 2 et 3 sont des capteurs, le nœud 4 est à 90 m, hors de portée. Une flèche apparaît à chaque émission. LED : rouge = « j'émets », vert = « j'ai reçu », bleu = « alerte ».
Mote output	Les capteurs écrivent `TX temp=…`. Le puits écrit `RX noeud=2 temp=…`, puis `>>> ALERTE SURCHAUFFE` au-dessus de 90 °C.
Radio messages	Chaque paquet radio qui passe dans l'air.

> Le nœud 4 émet bien (on voit ses `TX`), mais le puits ne reçoit **jamais** ses mesures : il est trop loin. Dans le vrai projet, ce problème se résout avec le routage multi-saut, où un nœud relaie les messages d'un autre.

Variante · Charger le binaire à la main

C'est la manipulation qu'on demande en TP.

```
cd ~/contiki-ng/tools/cooja
./gradlew run
```

1. **File** → **New simulation**, garde les réglages par défaut, puis **Create**.
2. **Motes** → **Add motes** → **Create new mote type** → **Sky mote**.
3. **Browse**, puis choisis `~/projets/01_demo_tinyos_cooja/build/telosb/main.exe`, puis **Create**.
4. Nombre de cartes : **4**, puis **Add motes**.
5. Dans la fenêtre **Network**, fais glisser une carte loin des autres.
6. Clique sur **Start**.

Facultatif · Test automatique sans fenêtre

```
cd ~/projet_g7/simulation/01_demo_tinyos_cooja
bash tester.sh
```

La fin doit afficher le nombre de mesures reçues par le puits, `alerte=true`, puis `TEST OK`.

Comprendre le code

FICHIER	RÔLE
`DemoUsineAppC.nc`	Le **câblage** : on déclare des briques (minuterie, radio, LED, port série) et on les relie avec des flèches `->`.
`DemoUsineC.nc`	Le **comportement**. TinyOS n'a pas de boucle principale : on écrit des réactions à des événements (`Boot.booted` , `Minuterie.fired` , `Receive.receive`).
`DemoUsine.h`	Le **format du message radio** (n° du nœud, n° de séquence, température) et les réglages (`PERIODE_MS` , `SEUIL_ALERTE`).
`demo_usine.csc`	Le **scénario Cooja** : position des 4 cartes, portée radio (50 m), fenêtres à afficher.

Le même programme tourne sur les 4 cartes. Chacune lit son numéro, attribué par Cooja, pour savoir si elle est le puits (n° 1) ou un capteur. Pour expérimenter, modifie `PERIODE_MS` ou `SEUIL_ALERTE` dans `DemoUsine.h`, puis relance les étapes 2 et 3.

En cas de problème

SYMPTÔME	SOLUTION
`make: *** No rule to make target` ou `Makefile.include` introuvable	`TINYOS_ROOT_DIR` n'est pas défini : refais la fin de l'étape 0, puis ouvre un nouveau terminal.
`nescc: command not found`	Le compilateur nesC manque : voir l'annexe.
`Cooja introuvable`	Contiki-NG n'est pas dans `~/contiki-ng`. Indique son emplacement : `export COOJA_DIR=/chemin/vers/contiki-ng/tools/cooja`.
La fenêtre Cooja ne s'ouvre pas, ou le terminal reste bloqué sans rien afficher	Un autre Cooja est peut-être déjà lancé avec Gradle. Ferme-le, puis relance.
`bash\r: No such file`	Les scripts ont gardé les fins de ligne Windows : refais la commande `sed` de l'étape 1.

Annexe · Installer les outils sur Ubuntu

> À faire uniquement si l'étape 0 a signalé un outil manquant. Ces commandes ont besoin d'Internet et du mot de passe administrateur (`sudo`). Le premier lancement de Cooja télécharge aussi ses dépendances.

```
# Outils de base, Java, compilateurs MSP430 et nesC
sudo apt update
sudo apt install -y git make python3 openjdk-17-jdk \
     gcc-msp430 binutils-msp430 msp430-libc msp430mcu nescc

# TinyOS
git clone https://github.com/tinyos/tinyos-main.git ~/tinyos-main
echo 'export TINYOS_ROOT_DIR=$HOME/tinyos-main' >> ~/.bashrc
source ~/.bashrc

# Contiki-NG, qui contient Cooja
git clone --recursive https://github.com/contiki-ng/contiki-ng.git ~/contiki-ng
cd ~/contiki-ng/tools/cooja && ./gradlew run   # premier lancement : télécharge les dépendances
```

Si un paquet est introuvable sur ta version d'Ubuntu, note le message exact et envoie-le : la solution dépend de la version.